Product Design

Team Members:

Rowland Foster

Rishav R. Dahal

Julian Bertrand

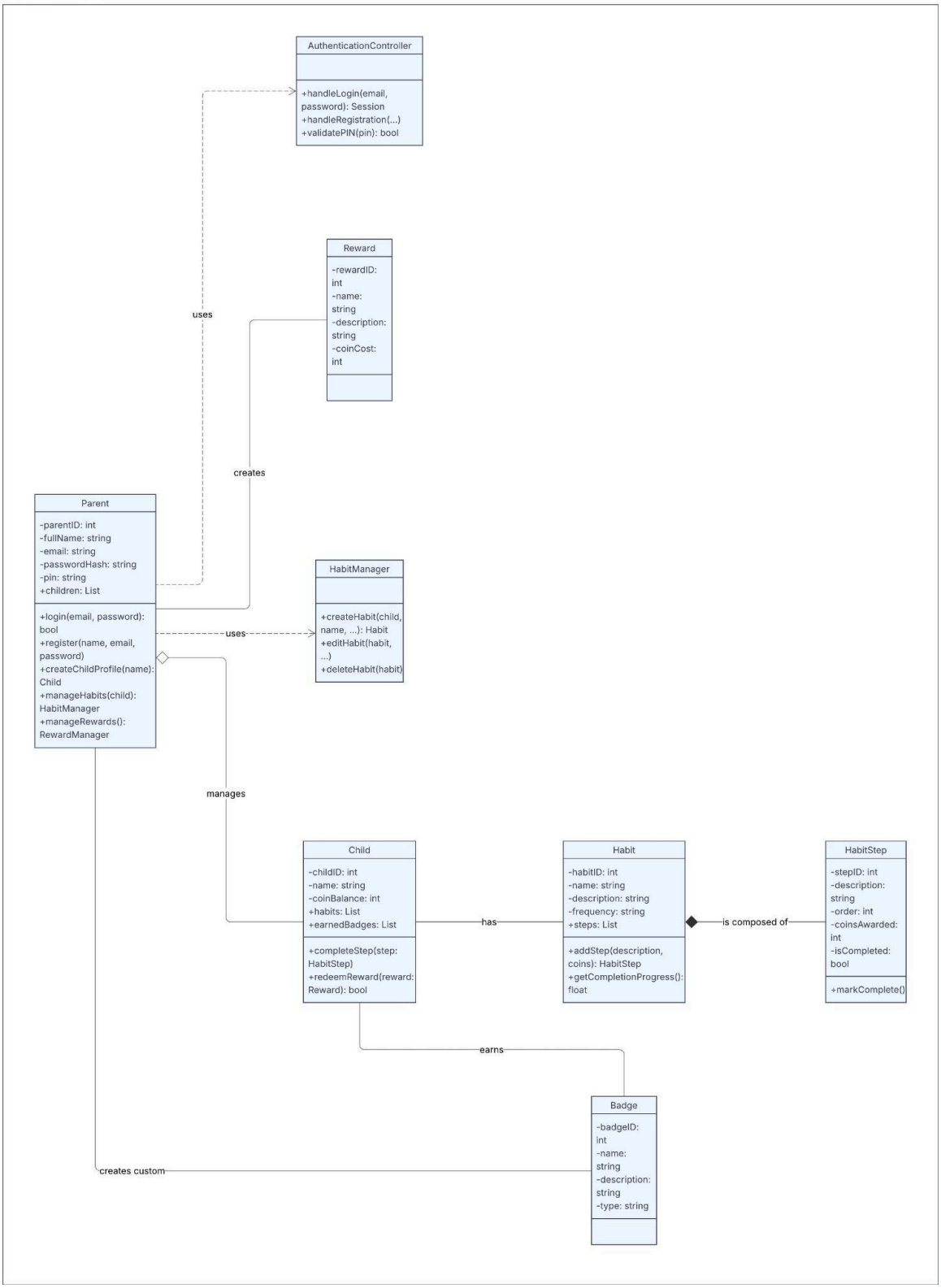
Abyan Haq

Derain Mendez

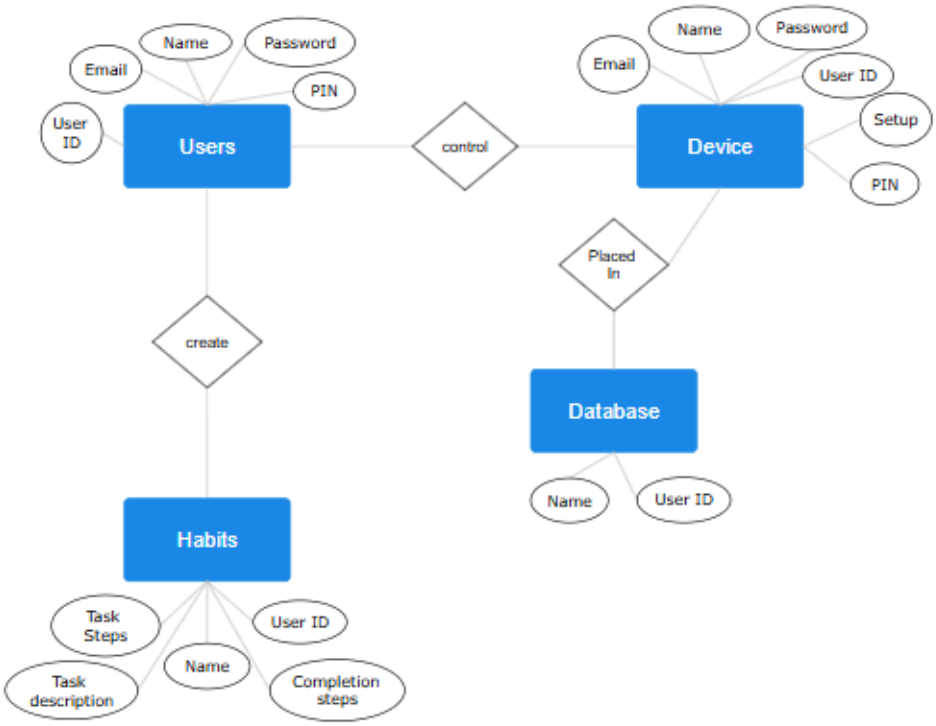
Revision Number	***Revision Date***	***Summary of Changes***	***Author(s)***
1	*10/08/2025*	*Added initial versions of the class diagram, ER diagram, IA diagram, user interface wireframes, design summary, and design rationale.*	*Rishav R. Dahal, Rowland Foster, Derian Mendez, Julian Bertrand, Abyan Huq*

Class Diagram

Compact UML Class Diagram

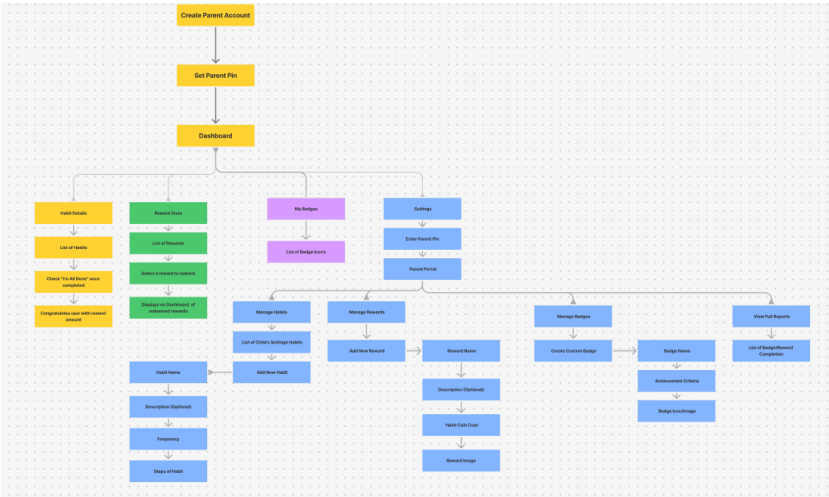


ER Diagram



Information Architecture Diagram

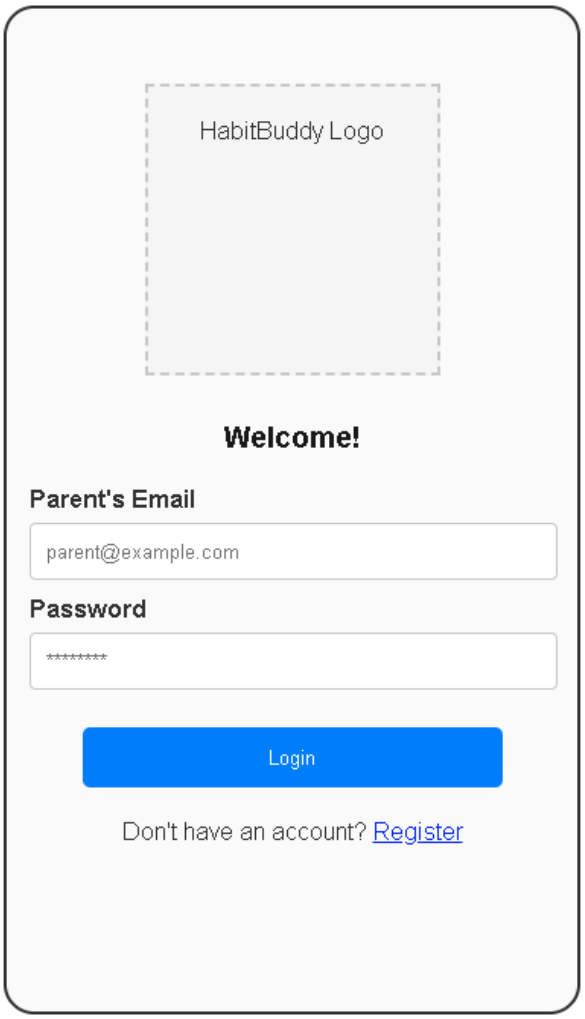
[Link to IA Diagram]



User Interface Wireframe(s)/Screenshot(s)

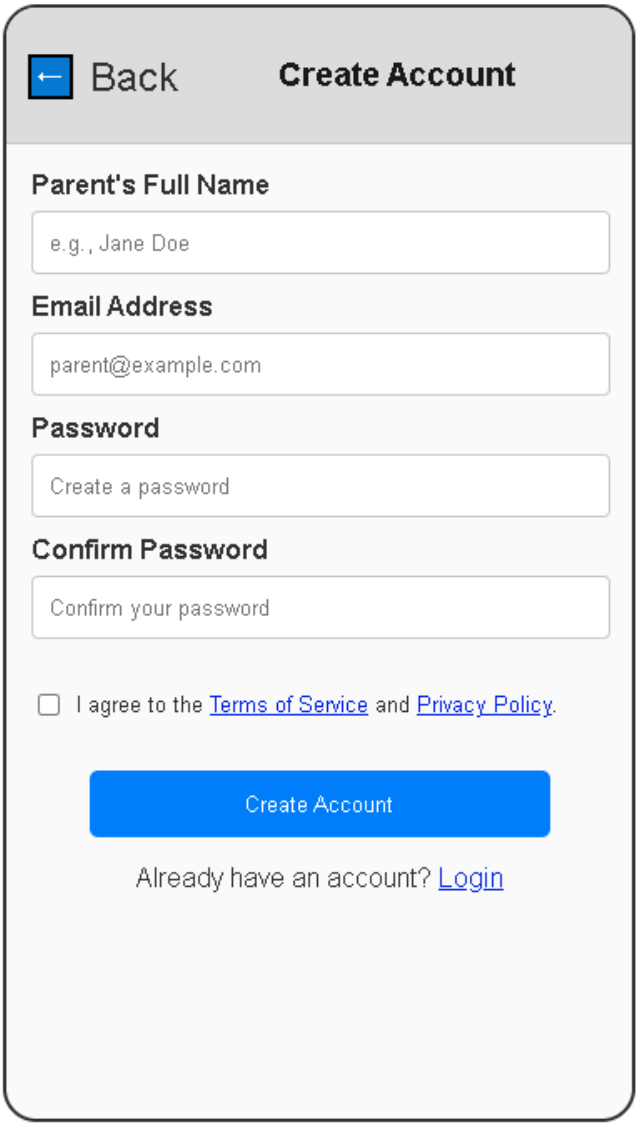
The following wireframes are designed to illustrate the overall structure and functionality of our app. The appearance of the final product will be a more refined version of the screens shown below.

ScreenID: SCR_1 - Login/Register



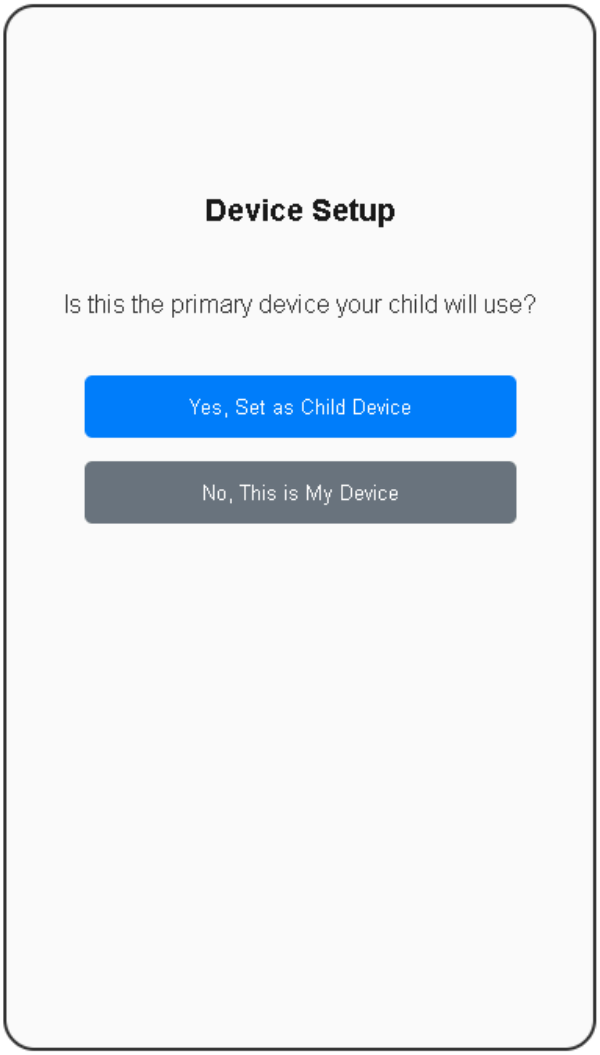
This is the initial screen for the parent to login or register. Due to child data protection laws, there is no separate login for the child. The parent will login on their child's device and set the device as the child device. This ensures that minimal personal information of the child is collected.

ScreenID: SCR_1A - Parent Account Registration



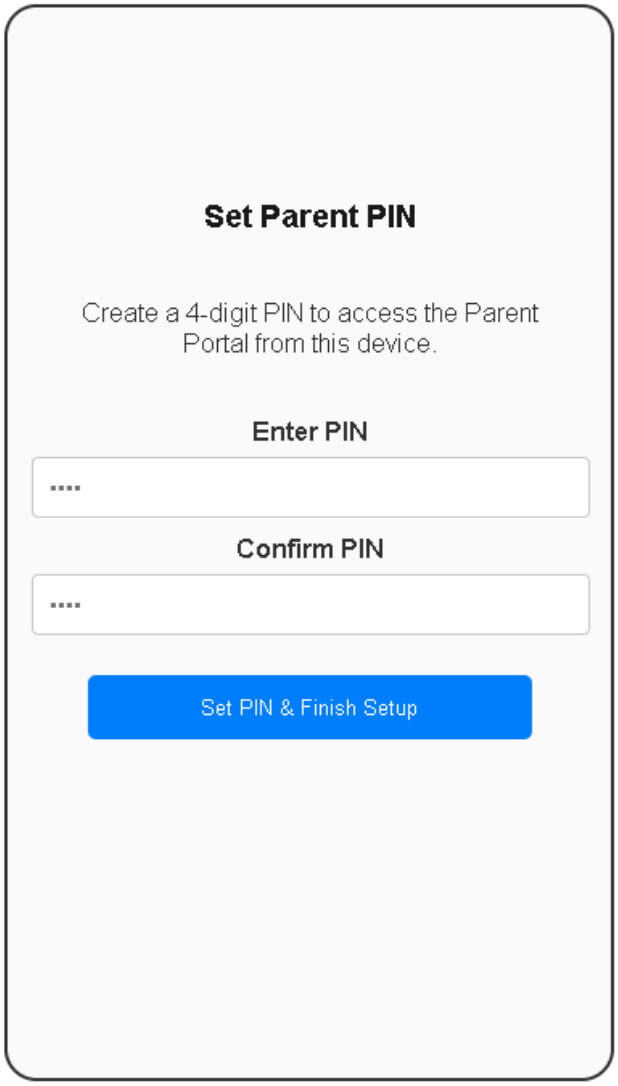
This screen allows the parent to create their account. The “Confirm Password” section ensures that the user has typed their password correctly, preventing login issues later if they mistyped their password during the account creation process.Agreement to the TOS and privacy policy ensures that the app adheres to privacy laws and app store rules.

ScreenID: SCR_2 - Initial Device Setup



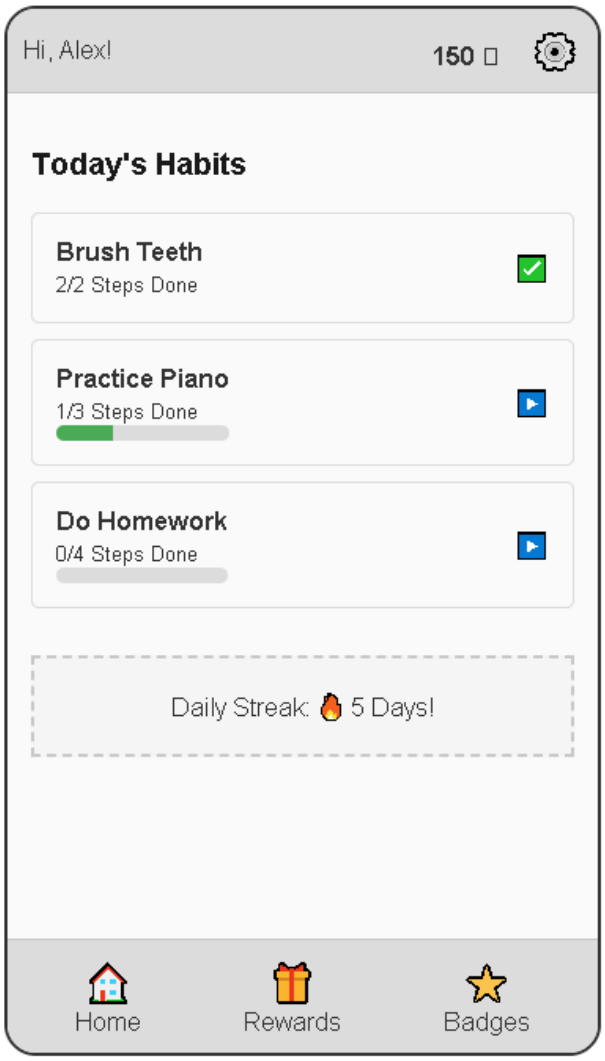
After the parent signs in, the app will prompt them to designate the device as the parent device or the child device. Selecting “Yes” will lock the device into child mode, prompting the parent to set a PIN. Selecting “No” would set the device to parent mode.

ScreenID: SCR_3 - Set Parent PIN



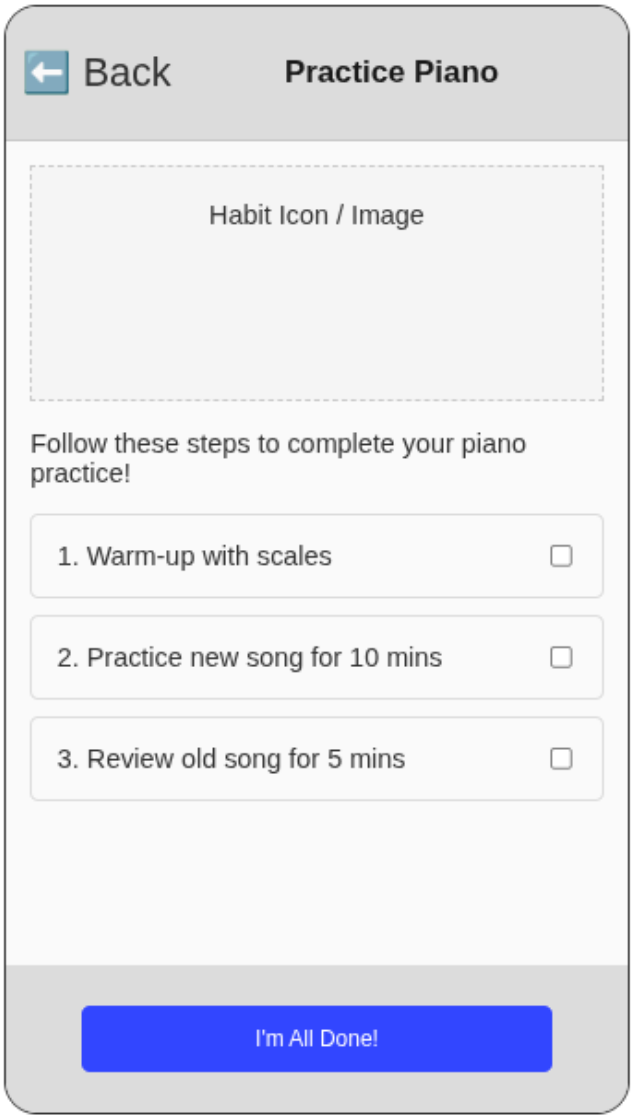
If this is the child device, the parent will create a secure PIN to protect the parent-only sections of the app. The PIN is required every time the parent wants to access the parent dashboard from the child device. Although this may not be necessary that often, this is the best solution we found to prevent the child from having to create their own account.

ScreenID: SCR_4 - Child Dashboard Home



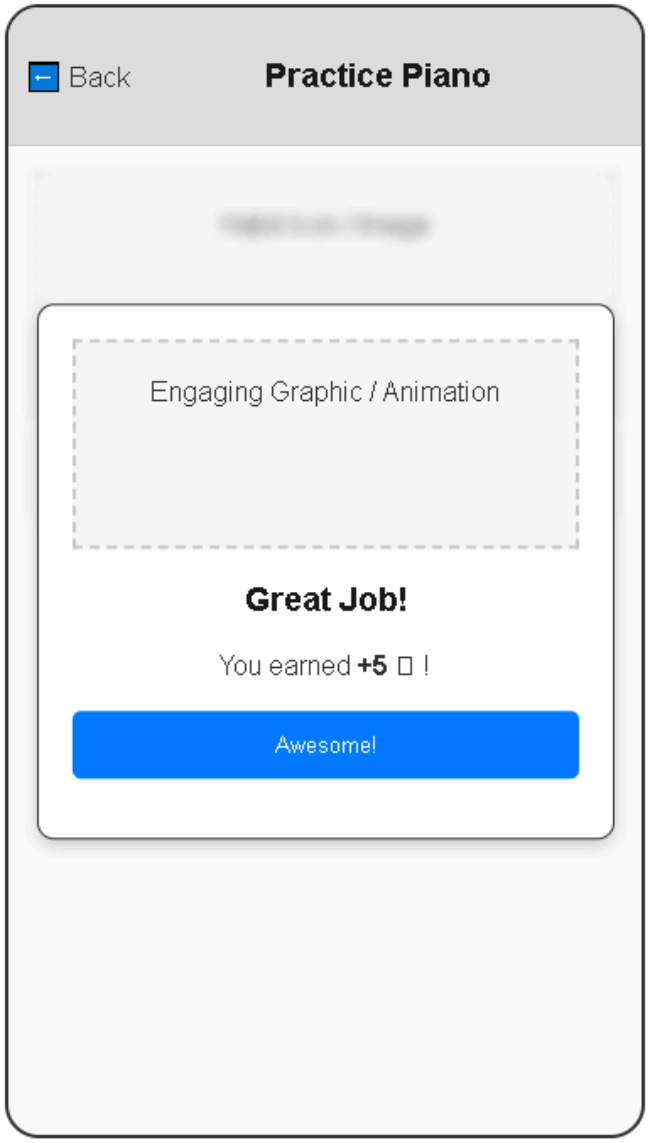
This is the main dashboard for the child view, showing their daily tasks, progress for each task, task completion streak, and HabitCoin balance. The navigation bar includes options for the home screen (current page), the reward store, and the badge gallery.

ScreenID: SCR_5 - Habit Detail and Steps



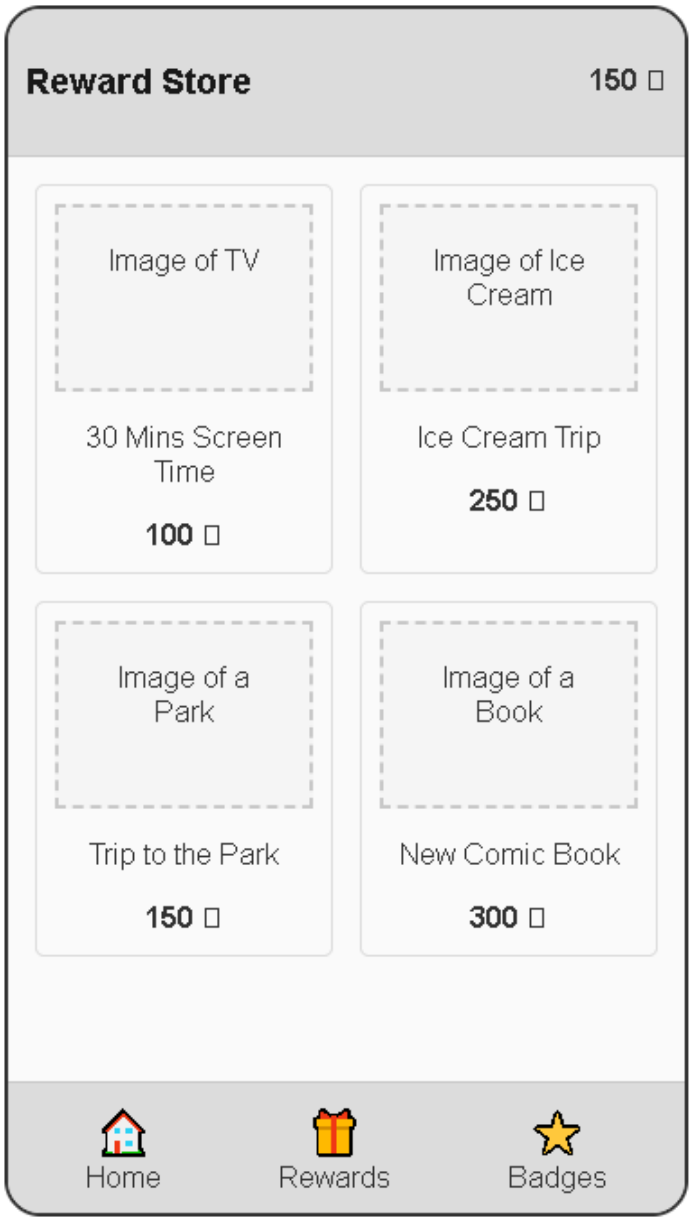
This is the view for an individual task, showing each step for completion. The child will check off each subtask when completed, and then hit the “I’m All Done” button to complete the overall task.

ScreenID: SCR_6 - Habit Completion



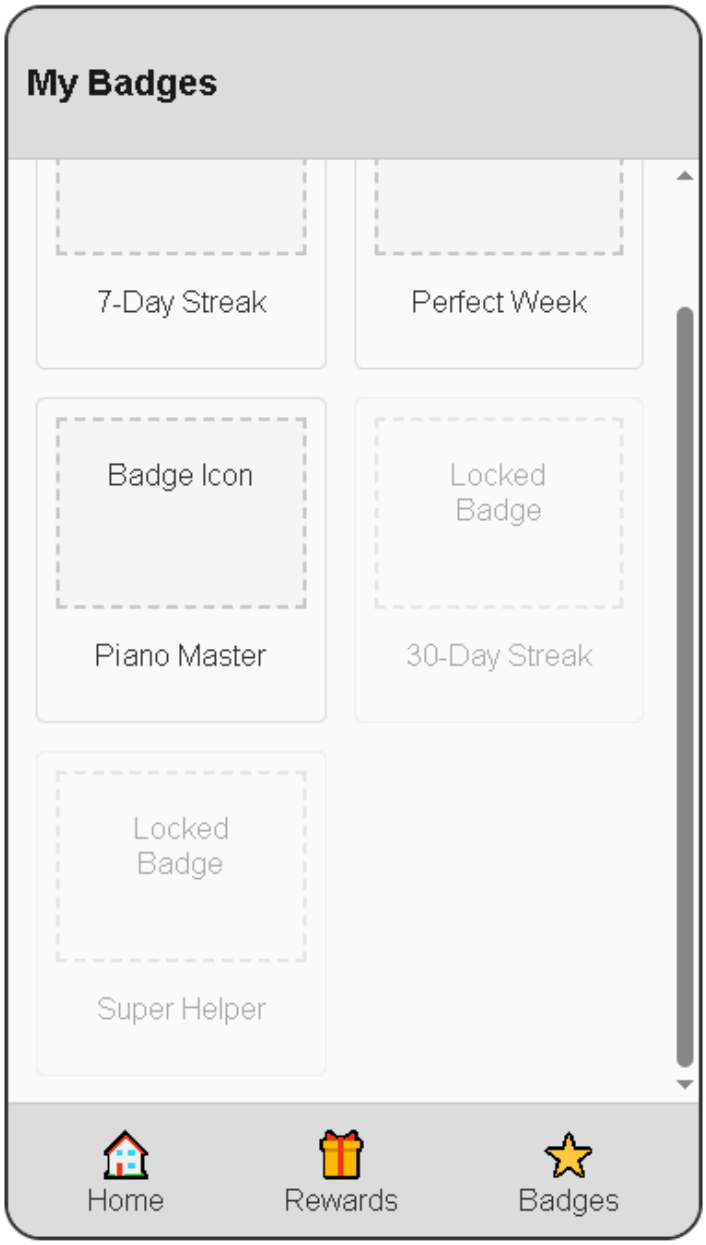
This screen provides a positive message upon task completion. It also communicates the amount of coins earned for completing the task.

ScreenID: SCR_7 - Reward Store



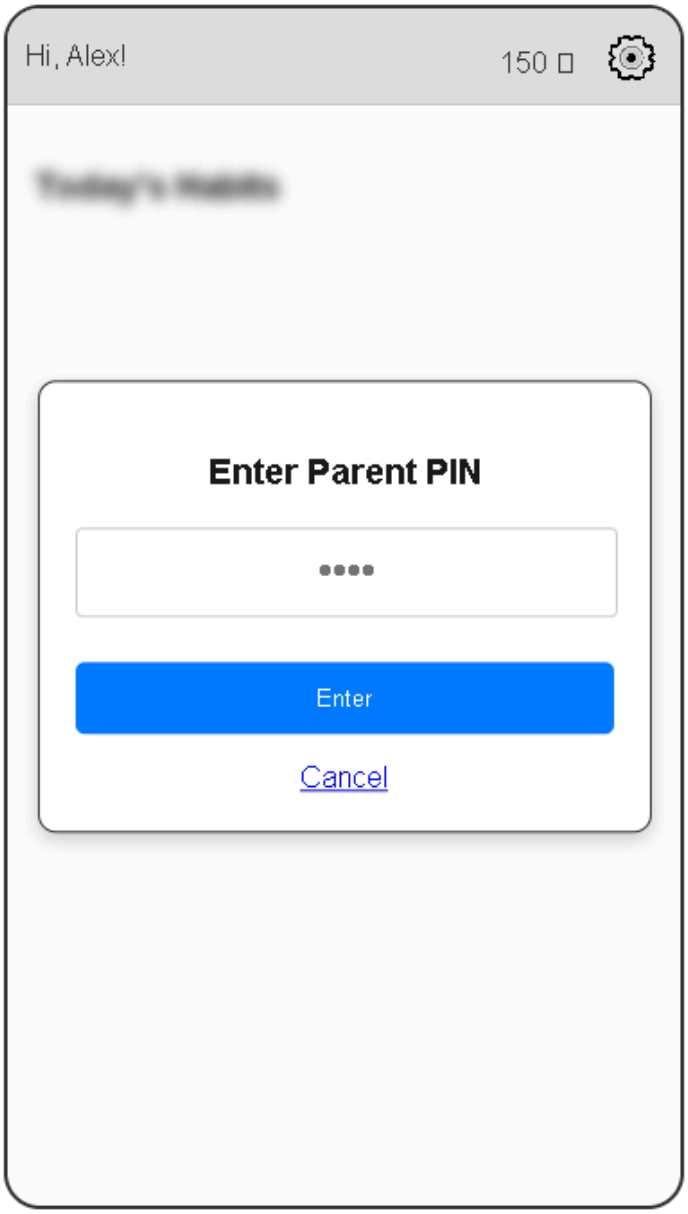
This screen allows the child to use their earned coins on parent-configured rewards. These rewards can be customized, including what the reward is, and the amount of coins to redeem it.

ScreenID: SCR_08 - Badge Gallery



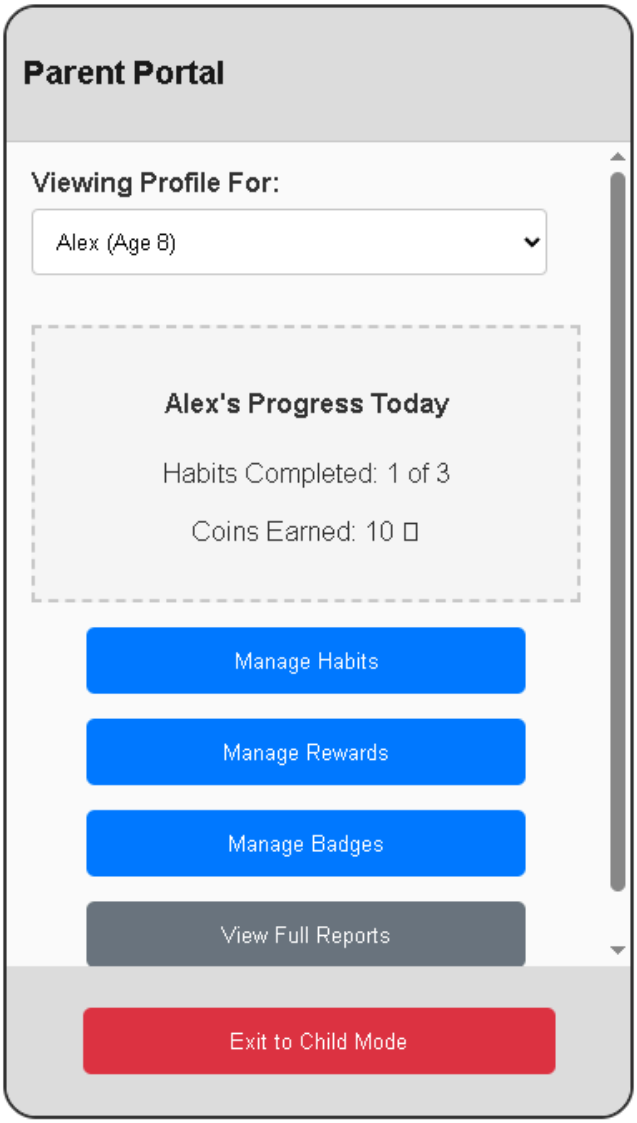
This is the screen where the child can view all the digital badges they have earned. These can be configured by the parent to personalize them for their child. Unearned badges are shown as greyed out. Tapping on the badge would show more details as to how the badge was earned.

ScreenID: SCR_9 - Parent PIN Entry



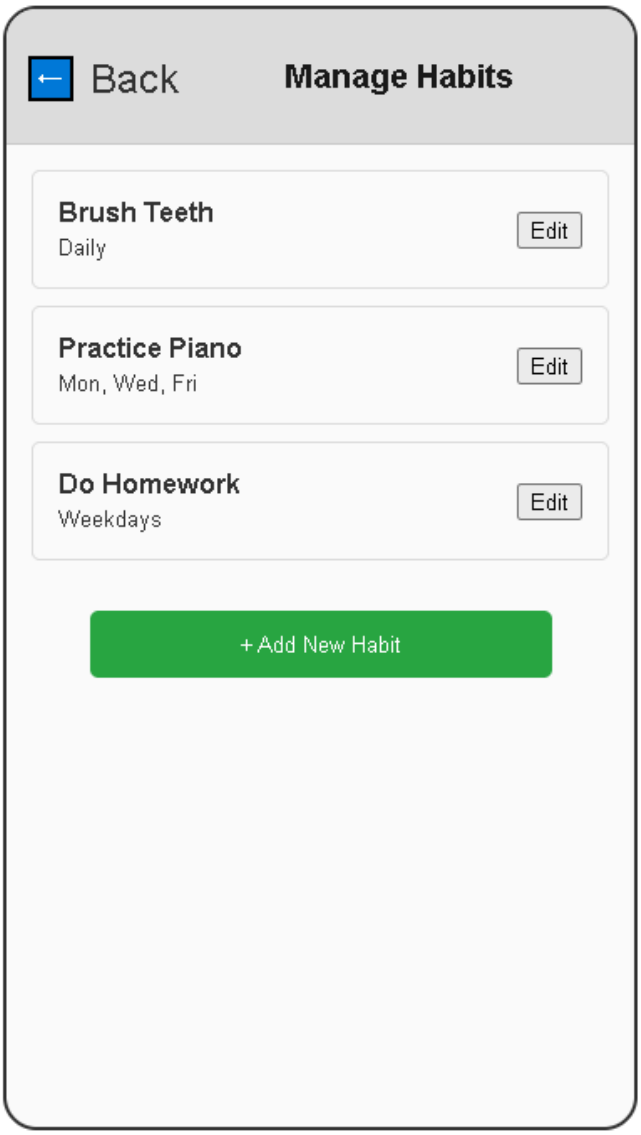
This screen appears when the setting icon is tapped on the child view. It prevents the child from accessing the parent portal/settings. A correct PIN will lead to the parent dashboard, and an incorrect PIN will show an error message.

ScreenID: SCR_10 - Parent Dashboard



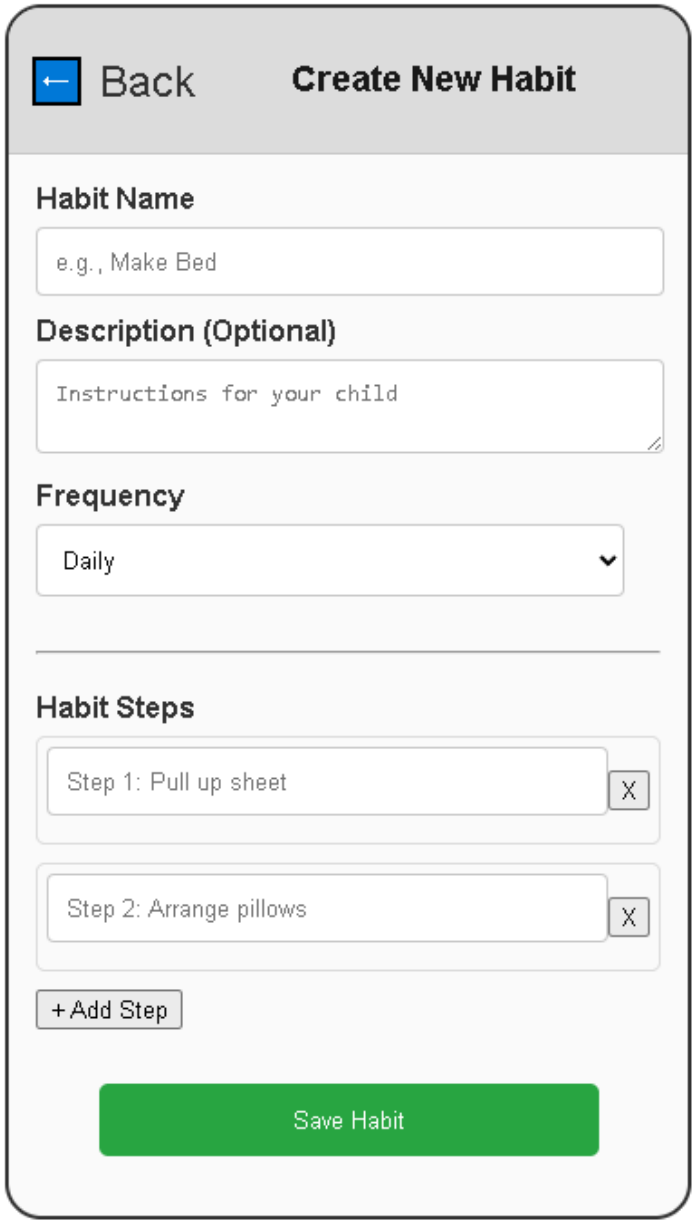
This is the main dashboard for managing the child's habits, rewards, badges, and to view their overall progress. It also has multi-child functionality, allowing the parent to have separate dashboards for each child.

ScreenID: SCR_11 - Manage Habits



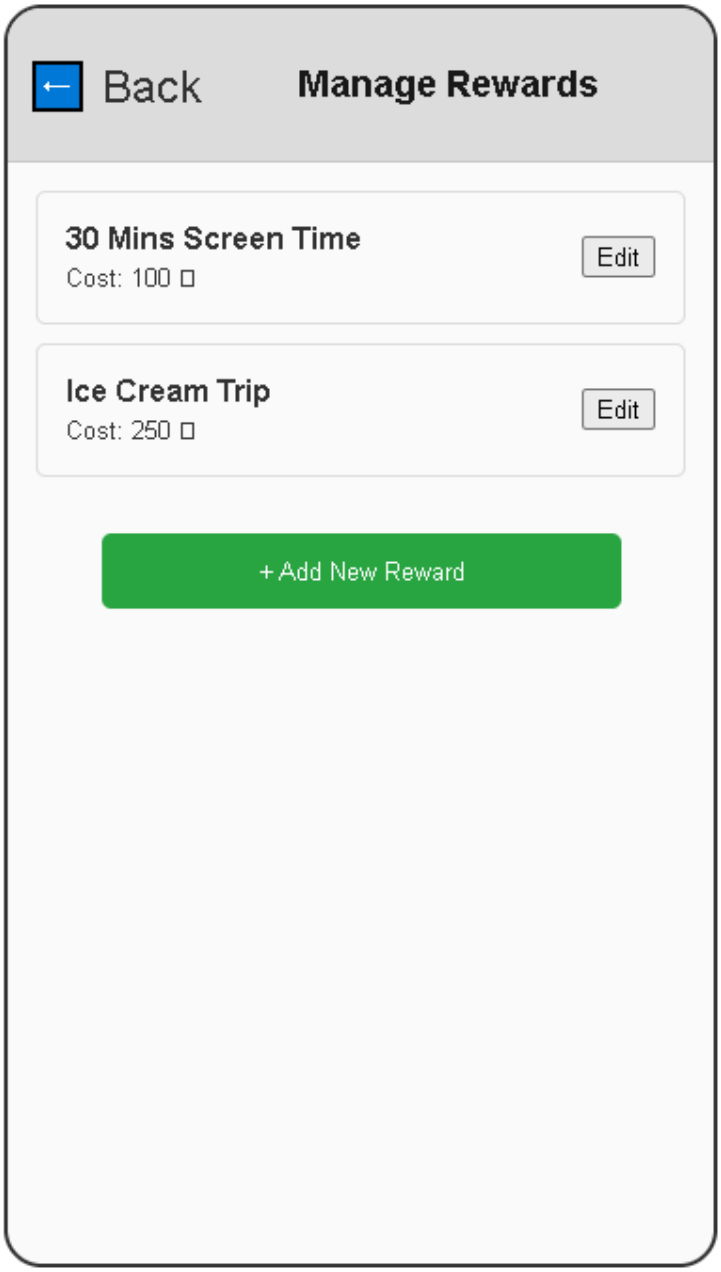
This is a list view of the child's habits, allowing them to edit habits, or add new ones.

ScreenID: SCR_12 - Create/Edit Habit



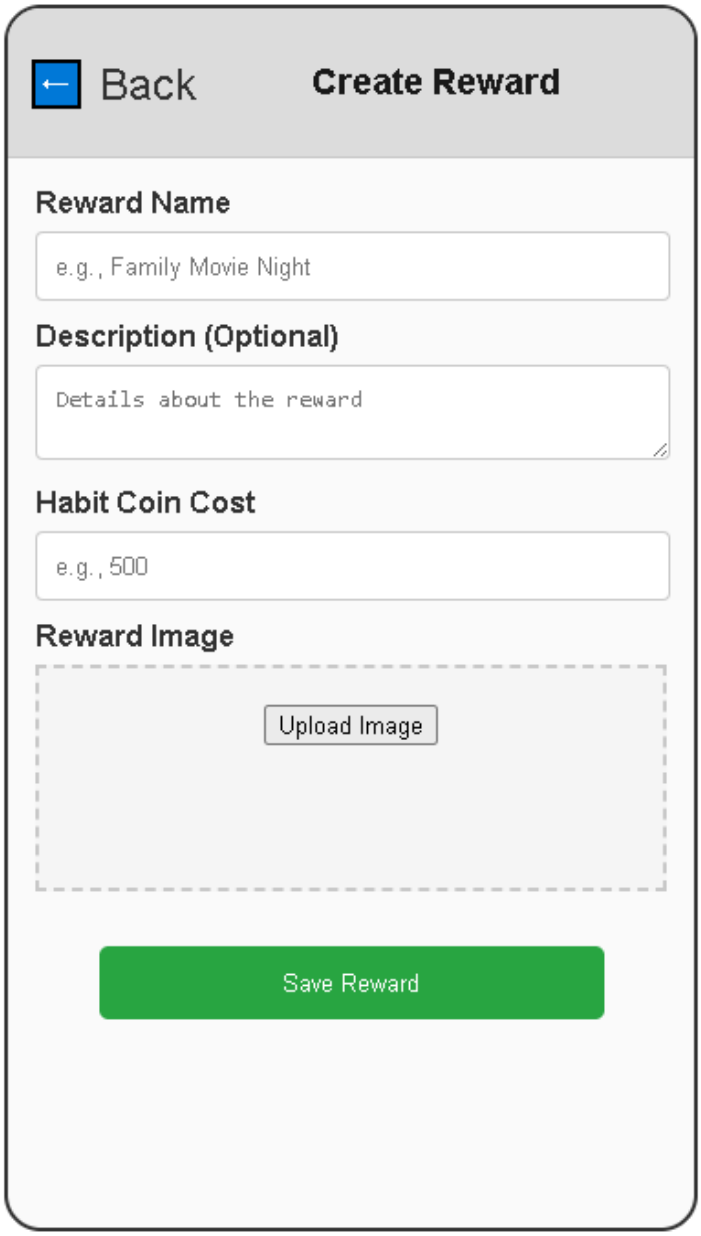
This screen allows the parent to create a new habit with a name, description, frequency, and habit steps. When editing an existing habit, the form would look the same but with the information already filled out, allowing the parent to change the different options.

ScreenID: SCR_13 - Manage Rewards



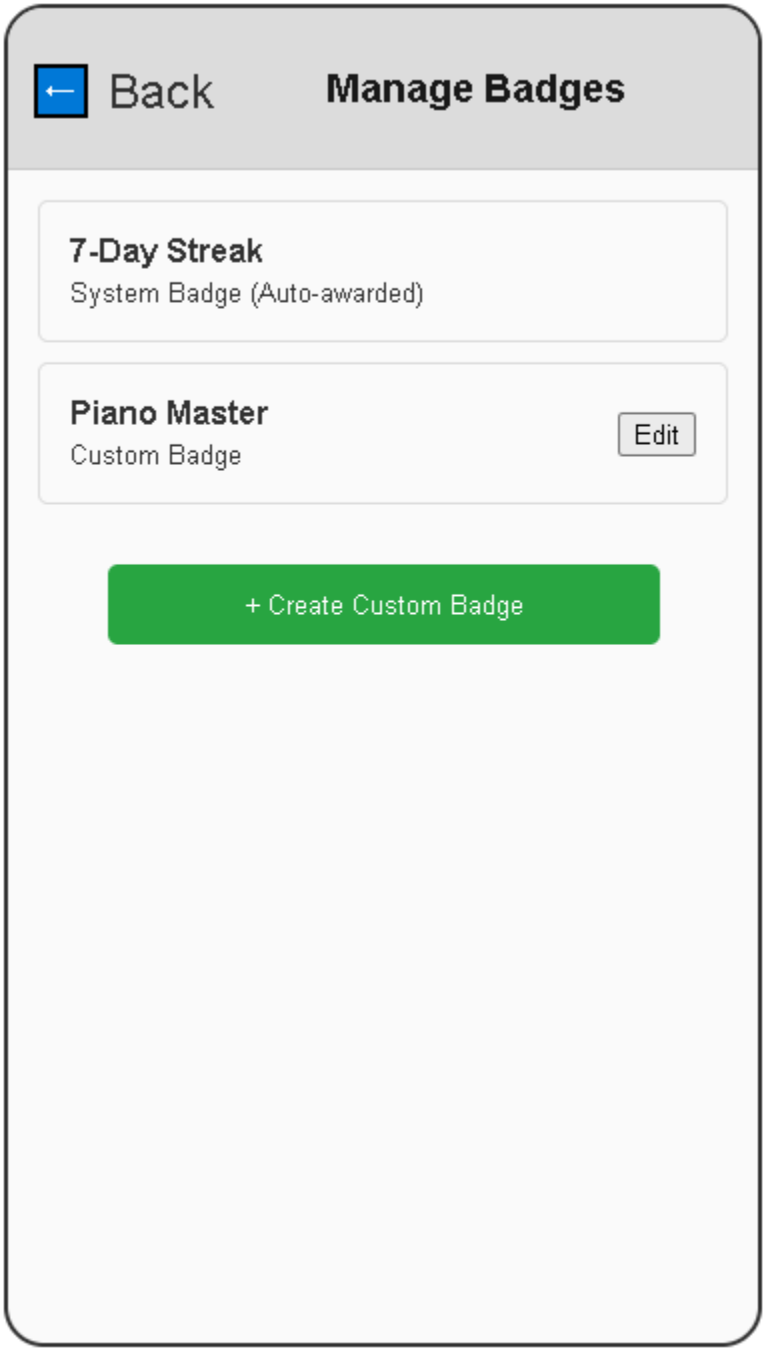
This screen allows the parent to edit and create new rewards for their child.

ScreenID: SCR_14 - Create/Edit Reward



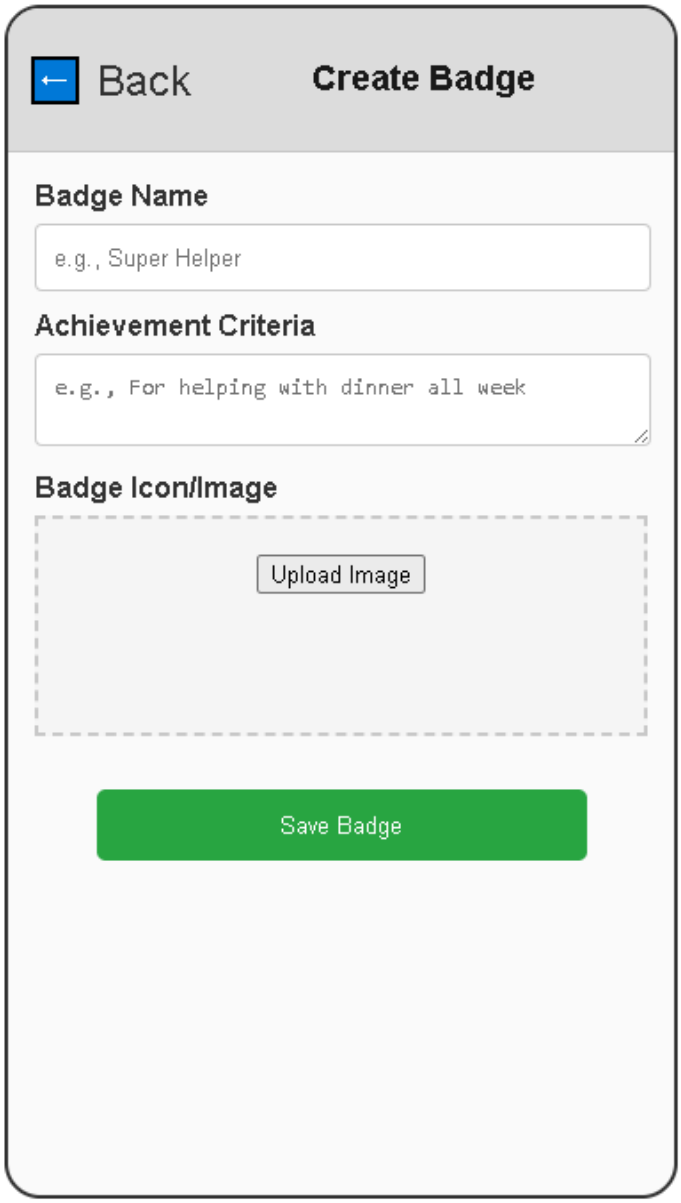
This form allows parents to create rewards for their child and set their cost in habit coins. The form for editing an existing reward looks the same, but with the information pre-filled.

ScreenID: SCR_15 - Manage Badges



This screen shows a list of badges that the child can earn. It also has the option to create new custom badges. The app will have default badges such as habit streaks, but the parent will be able to create personalized badges as well.

ScreenID: SCR_16 - Create Custom Badge



This form allows the parent to create personalized badges for their child based on what habits they are working on. For now, these custom badges are manually awarded, but the design of this system may change to allow the parent to set certain habit milestones that will allow for the badge to be automatically awarded.

Design Summary

Our design for the HabitBuddy app is a simple and practical mobile web application built to serve both parents and children from a single application. The parent is responsible for logging in, creating habits, and managing rewards through a straightforward, PIN-protected setting area. The child's view is designed to be visual and easy to use, allowing them to see their daily tasks, check them off, and view their progress and rewards. This single-app approach keeps the project focused, and more manageable than any other solution.

Our plan is to keep the code organized by separating different functions into their own classes, making it easier for our team to build and test features at the same time (agile methodology). To support the features like daily streaks and progress tracking, our database will log these actions such as when a child completes a step, providing a reliable history of their activity on the app. This approach was chosen because it is achievable for our team to build in the given timeframe while still meeting the goals of the project.

Design Rationale

This section explains the main choices our team made for the app's design and why we made them. We focused on what was practical for our capstone project timeline and requirements.

1. One App for Everyone (Instead of Two): We decided to build one single application for both parents and children. The main reason was to keep the project scope manageable for our team. Building and connecting two separate apps would have been too complex and time-consuming for the scope of this project. This approach is also simpler for families, as they only need to install and manage one app.
2. Simple PIN for Parent Mode: To access the parent settings on a child's device, we are using a 4-digit PIN instead of making the parent type their full password again. This was a practical choice. The main goal is to stop a young child from

accidentally changing settings, and a PIN is good enough for that. It's much easier for a parent to use quickly, and it's simpler for our team to implement than a full re-authentication system.

3. Detailed Database Logging for Progress: For tracking progress, our database will log every time a child completes a single step of a habit. We chose this design because it’s the only reliable way to track daily streaks, which is a key feature in our requirements. The simpler alternative was to just have a status like "completed" on the habit itself, but we realized that would make calculating streaks correctly more difficult. This design is more work upfront but prevents a major headache later in the project.
4. Organizing Code into Separate Parts: We are splitting our code into smaller, focused classes (like a HabitManager and an AuthenticationController) instead of putting all the functions inside one giant Parent class. This helps our team work on different features at the same time without causing issues for one another. It keeps the code cleaner and makes it much easier for us to find and fix bugs, which is important for finishing the project on time.